

=====

CO1_ASSESSMENT TOOL 1

APPLICATION - BASED QUESTION_2Q

=====

Submitted in partial fulfilment for the course of

CSA0606-Design and Analysis of Algorithm

for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

By, Avinash M

(192521196)

UNDER THE GUIDANCE OF

Dr. Rajagopal.K



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

July 2026

Questions:**Q5. Application: Financial Analytics System**

A financial system uses an algorithm with recurrence $T(n) = 2T(n/2) + n \log n$. Apply the Master Theorem and determine the complexity. Compare $f(n)$ with $n^{\log_b a}$ and justify the correct case. Analyze performance for large datasets. Interpret efficiency clearly.

Q45. Application: Smart Grid Fault Detection System

A power grid monitoring system uses the recurrence $T(n) = 3T(n/2) + n^2$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze performance for large-scale grid systems. Interpret efficiency using asymptotic notation.

Question 5

Application: Financial Analytics System

Aim

To determine the time complexity of the recurrence relation $T(n) = 2T(n/2) + n \log n$ using the Master Theorem, and to analyze the efficiency of the algorithm used in a financial analytics system that processes large volumes of transaction data.

Theory

Financial analytics systems frequently use divide-and-conquer algorithms to process large datasets such as stock prices, transaction logs, and risk models. Such algorithms split the problem into smaller sub-problems, solve them recursively, and combine the results. The given recurrence is:

$$T(n) = 2T(n/2) + n \log n$$

where:

- Number of sub-problems, $a = 2$
- Size of each sub-problem $= n/2$, so $b = 2$
- Extra work done to combine/merge results, $f(n) = n \log n$

The Master Theorem is applied to recurrences of the standard form $T(n) = aT(n/b) + f(n)$, and it determines the growth of $T(n)$ by comparing $f(n)$ with $n^{(\log_b a)}$.

Step 1: Identify Parameters

Given $T(n) = aT(n/b) + f(n)$, comparing with $T(n) = 2T(n/2) + n \log n$ gives:

- $a = 2$
- $b = 2$
- $f(n) = n \log n$

Step 2: Compute $n^{(\log_b a)}$

$$\log_b a = \log_2 2 = 1$$

$$n^{(\log_b a)} = n^1 = n$$

Step 3: Compare $f(n)$ with $n^{(\log_b a)}$

We now compare:

$$f(n) = n \log n \text{ versus } n^{(\log_b a)} = n$$

Here, $f(n) = n \log n$ is asymptotically larger than n , since the $\log n$ factor grows without bound. However, $f(n)$ is not polynomially larger than n ($n \log n$ is not $\Omega(n^{(1+\epsilon)})$ for any $\epsilon > 0$). This means the recurrence does not fit exactly into the standard Case 1, Case 2, or Case 3 of the basic Master Theorem, and the extended (generalized) version of the Master Theorem must be used.

Step 4: Apply the Extended Master Theorem

The extended Master Theorem states that if $f(n) = \Theta(n^{(\log_b a)} \cdot \log^k n)$ for some $k \geq 0$, then:

$$T(n) = \Theta(n^{(\log_b a)} \cdot \log^{(k+1)} n)$$

In this problem, $f(n) = n \log n = n^1 \cdot \log^1 n$, which matches the form $n^{(\log_b a)} \cdot \log^k n$ with $k = 1$. This corresponds to the extended form of Case 2 of the Master Theorem (the boundary case, generalized for logarithmic factors). Substituting $k = 1$:

$$T(n) = \Theta(n \cdot \log^2 n)$$

Time Complexity

$$T(n) = \Theta(n \log^2 n)$$

Growth Rate Table

Dataset Size (n)	n	$n \log^2 n$
1,000	1,000	$\approx 99,300$
10,000	10,000	$\approx 1,767,000$
100,000	100,000	$\approx 27,600,000$
1,000,000	1,000,000	$\approx 397,000,000$

Performance Analysis

In a financial analytics system, this recurrence typically models algorithms that merge partially sorted or partially processed transaction records (similar to merge-sort-style aggregation) while performing additional $O(\log n)$ weighted work per merge step, such as interest computation, risk-weighting, or time-series alignment.

- The growth rate $\Theta(n \log^2 n)$ is only slightly worse than $\Theta(n \log n)$ (used in standard merge sort).
- It scales far better than quadratic algorithms such as $O(n^2)$.
- It remains practical even for very large financial datasets containing millions of transaction records.

Efficiency Analysis

Advantages:

- Efficient for large-scale financial data processing.
- Only marginally slower than the optimal $\Theta(n \log n)$ divide-and-conquer bound.
- Well suited for real-time or near-real-time analytics pipelines.

Disadvantages:

- Slightly higher overhead than plain $O(n \log n)$ algorithms due to the extra

logarithmic factor.

- Requires careful implementation of the merge/combine step to actually achieve $n \log n$ work per level.

Conclusion

Applying the Master Theorem (extended form) to $T(n) = 2T(n/2) + n \log n$ gives a time complexity of $\Theta(n \log^2 n)$. Since $f(n) = n \log n$ grows at the same polynomial order as $n^{(\log_b a)} = n$ but with an extra logarithmic factor, the recurrence falls under the boundary/extended Case 2 of the Master Theorem. This complexity is efficient and scalable, making the algorithm well suited for large-scale financial analytics systems that must process massive volumes of transactional data quickly and reliably.

Source Code (C Program)

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int n;
    cout << "Enter dataset size: ";
    cin >> n;

    double complexity = n * pow(log2(n), 2);

    cout << endl << "Master Theorem Analysis" << endl;
    cout << "-----" << endl;
    cout << "Recurrence:  $T(n) = 2T(n/2) + n \log n$ " << endl;
    cout << "Time Complexity =  $\Theta(n \log^2 n)$ " << endl;
    cout << "Estimated Growth = " << complexity << endl;

    return 0;
}
```

Output:

```
Output
Enter dataset size: 10000

Master Theorem Analysis
-----
Recurrence:  $T(n) = 2T(n/2) + n \log n$ 
Time Complexity =  $\Theta(n \log^2 n)$ 
Estimated Growth = 1.76563e+06

=== Code Execution Successful ===
```

Question 45

Application: Smart Grid Fault Detection System

Aim

To determine the time complexity of the recurrence relation $T(n) = 3T(n/2) + n^2$ using the Master Theorem, and to analyze the scalability and efficiency of the fault detection algorithm used in a smart power grid monitoring system.

Theory

Smart grid monitoring systems continuously analyze sensor data from thousands of nodes across the power network to detect faults, overloads, and anomalies. Such fault-detection algorithms often divide the grid into smaller sub-regions, analyze each recursively, and then combine the results using a comparison or aggregation step. The given recurrence is:

$$T(n) = 3T(n/2) + n^2$$

where:

- Number of recursive sub-problems, $a = 3$
- Size of each sub-problem $= n/2$, so $b = 2$
- Extra work done at each level to combine/analyze results, $f(n) = n^2$

The Master Theorem compares $f(n)$ with $n^{\log_b a}$ to determine which of the three standard cases applies.

Step 1: Identify Parameters

Given $T(n) = aT(n/b) + f(n)$, comparing with $T(n) = 3T(n/2) + n^2$ gives:

- $a = 3$
- $b = 2$
- $f(n) = n^2$

Step 2: Compute $n^{\log_b a}$

$$\log_b a = \log_2 3 \approx 1.585$$

$$n^{\log_b a} = n^{1.585} \text{ (approximately)}$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$

We now compare:

$$f(n) = n^2 \text{ versus } n^{\log_b a} = n^{1.585}$$

Since n^2 grows faster than $n^{1.585}$, $f(n)$ is polynomially larger than $n^{\log_b a}$. Specifically, $n^2 = n^{(1.585 + 0.415)}$, so $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon \approx 0.415 > 0$. This satisfies the condition required for Case 3 of the Master Theorem.

Regularity condition check for Case 3: $a \cdot f(n/b) \leq c \cdot f(n)$ for some constant $c < 1$ and

sufficiently large n .

$$3(n/2)^2 = (3/4)n^2 = 0.75 n^2$$

Since $0.75 n^2 \leq c \cdot n^2$ holds for $c = 0.75 < 1$, the regularity condition is satisfied.

Step 4: Apply the Master Theorem

Since $f(n)$ is polynomially larger than $n^{\log_b a}$ and the regularity condition holds, Case 3 of the Master Theorem applies:

$$T(n) = \Theta(f(n)) = \Theta(n^2)$$

Time Complexity

$$T(n) = \Theta(n^2)$$

Growth Rate Table

Grid Nodes (n)	$n^{1.585}$ (recursive part)	n^2 (combine work, dominant)
100	$\approx 1,516$	10,000
1,000	$\approx 48,000$	1,000,000
10,000	$\approx 1,517,000$	100,000,000
100,000	$\approx 47,900,000$	10,000,000,000

Performance Analysis

In a smart grid fault detection system, the n^2 term dominates the recurrence because the work done to combine and cross-check fault signals across sub-regions (for example, pairwise comparison of sensor readings) grows quadratically, while the recursive splitting contributes only the smaller $n^{1.585}$ term.

- As the number of monitored nodes (n) increases, execution time grows quadratically.
- For small to medium grids, performance remains acceptable.
- For very large, city-scale or national-scale grids, quadratic growth becomes a scalability bottleneck.

Efficiency Analysis

Advantages:

- Straightforward divide-and-conquer structure that is easy to implement and verify.
- Well suited for medium-sized grid segments where fault detection accuracy matters more than raw speed.

Disadvantages:

- Quadratic time complexity limits scalability for extremely large, nationwide smart grid networks.

- The combine step ($f(n) = n^2$) dominates and should be optimized (for example, using spatial partitioning) to improve real-world performance.

Conclusion

Applying the Master Theorem to $T(n) = 3T(n/2) + n^2$ shows that $f(n) = n^2$ is polynomially larger than $n^{(\log_b a)} = n^{1.585}$, and the regularity condition is satisfied. Therefore, Case 3 of the Master Theorem applies, giving a time complexity of $\Theta(n^2)$. This indicates that the fault detection algorithm's performance is dominated by the quadratic combine step rather than the recursive splitting. While acceptable for moderate grid sizes, this quadratic growth suggests that optimization of the combine step is important for scaling the fault detection system to very large power grids.

Source Code (C++ Program)

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int n;
    cout << "Enter number of grid nodes: ";
    cin >> n;

    double complexity = (double)n * n;

    cout << endl << "Master Theorem Analysis" << endl;
    cout << "-----" << endl;
    cout << "Recurrence:  $T(n) = 3T(n/2) + n^2$ " << endl;
    cout << "Time Complexity =  $\Theta(n^2)$ " << endl;
    cout << "Estimated Growth = " << complexity << endl;

    return 0;
}
```

OUTPUT:

Output

Enter number of grid nodes: 8

Master Theorem Analysis

Recurrence: $T(n) = 3T(n/2) + n^2$

Time Complexity = $\Theta(n^2)$

Estimated Growth = 64